🚀 AI Driven Software Engineering Bootcamp — Preparation Guide

Instructor: 7+ Years Web & Software Development Experience

Course Starts: May 1, 2026 | Time Left: ~2 Weeks

> **তোমার লক্ষ্য:** Frontend Developer থেকে Full-Stack Software Engineer হওয়া।
> **তোমার সুবিধা:** তুমি ইতিমধ্যে HTML, CSS, Bootstrap, Tailwind, JavaScript, React, Express.js, Node.js, MongoDB, Firebase এবং Basic Next.js জানো। এটা একটা বিশাল advantage!

📋 PREPARATION ROADMAP (Before May 1, 2026)

```
Week 1 (Apr 18-24): TypeScript Deep Dive (Mission 1 Prep)
Week 2 (Apr 25-30): SQL + Node/Express Refresh (Mission 2 & 3 Prep)
Bonus:              Prisma & Next.js Preview (Mission 4 Prep)
```

🎯 MISSION 1: Be a TypeScript Technocrat

কেন TypeScript শেখা জরুরি?

JavaScript is dynamic — it doesn't catch errors until runtime. TypeScript adds **static typing** on top of JavaScript, catching bugs at compile time. Every major company (Google, Microsoft, Airbnb) has moved to TypeScript.

Real-world use case: তুমি একটা React app বানাচ্ছো যেখানে একটা `user` object আছে। JavaScript-এ ভুল property access করলে runtime-এ error আসবে। TypeScript সেটা আগেই ধরে ফেলবে।

📚 TypeScript Core Concepts to Master

1. Basic Types

```typescript
```

```
// JavaScript (কোনো type safety নেই)
let age = 25;
age = "twenty five"; // ✅ No error in JS — কিন্তু bug!

// TypeScript (type safe)
let age: number = 25;
age = "twenty five"; // ❌ Error: Type 'string' is not assignable to type 'number'

// Common Types
let name: string = "Rahim";
let isActive: boolean = true;
let scores: number[] = [90, 85, 78];
let mixed: (string | number)[] = ["hello", 42]; // Union in array
```

2. Interfaces vs Types

```typescript
// Interface — Object এর shape define করতে
interface User {
  id: number;
  name: string;
  email: string;
  age?: number; // Optional field (? দিয়ে)
}

// Type Alias — More flexible
type Status = "active" | "inactive" | "pending"; // Union type
type ID = string | number;

// Real-world example: API response type করা
interface ApiResponse<T> {
  data: T;
  status: number;
  message: string;
}

// ব্যবহার:
const response: ApiResponse<User[]> = {
  data: [{ id: 1, name: "Karim", email: "karim@test.com" }],
  status: 200,
  message: "Success"
};
```

3. Generics — TypeScript-এর সবচেয়ে শক্তিশালী feature

```typescript
// Without generics — reusable নয়
function getFirstItem(arr: number[]): number {
  return arr[0];
}

// With generics — যেকোনো type-এর array-এ কাজ করে
function getFirstItem<T>(arr: T[]): T {
  return arr[0];
}

const firstNum = getFirstItem<number>([1, 2, 3]);   // 1
const firstName = getFirstItem<string>(["Rahim", "Karim"]); // "Rahim"

// Real-world: Generic API fetcher
async function fetchData<T>(url: string): Promise<T> {
  const res = await fetch(url);
  return res.json() as T;
}

// ব্যবহার:
const users = await fetchData<User[]>("https://api.example.com/users");
```

4. Enums

```typescript
// Real-world: Order status management
enum OrderStatus {
  PENDING = "pending",
  PROCESSING = "processing",
  SHIPPED = "shipped",
  DELIVERED = "delivered",
  CANCELLED = "cancelled"
}

function updateOrder(orderId: string, status: OrderStatus) {
  console.log(`Order ${orderId} is now ${status}`);
}

updateOrder("ORD-001", OrderStatus.SHIPPED); // ✅ Safe
updateOrder("ORD-001", "shipped");           // ❌ TypeScript error
```

5. Utility Types (Must Know!)

```typescript
interface Product {
  id: number;
  name: string;
  price: number;
  description: string;
}

// Partial — সব fields optional করে (update API-তে কাজে লাগে)
type UpdateProduct = Partial<Product>;
// { id?: number; name?: string; price?: number; description?: string }

// Pick — শুধু নির্দিষ্ট fields নাও
type ProductCard = Pick<Product, "id" | "name" | "price">;

// Omit — নির্দিষ্ট fields বাদ দাও
type CreateProduct = Omit<Product, "id">; // id ছাড়া (DB auto-generate করবে)

// Readonly — immutable করে দাও
type ReadonlyProduct = Readonly<Product>;

// Record — key-value map
type ProductInventory = Record<string, number>;
// { "laptop": 50, "phone": 100 }
```

6. TypeScript with React (তুমি এটা সরাসরি use করবে)

```typescript
```

```
// Component Props type করা
interface ButtonProps {
  label: string;
  onClick: () => void;
  variant?: "primary" | "secondary" | "danger";
  disabled?: boolean;
}

const Button: React.FC<ButtonProps> = ({ label, onClick, variant = "primary", disabl
  return (
    <button
      onClick={onClick}
      disabled={disabled}
      className={`btn btn-${variant}`}
    >
      {label}
    </button>
  );
};

// useState with type
const [users, setUsers] = useState<User[]>([]);
const [loading, setLoading] = useState<boolean>(false);

// Custom Hook with TypeScript
function useLocalStorage<T>(key: string, initialValue: T) {
  const [storedValue, setStoredValue] = useState<T>(() => {
    const item = localStorage.getItem(key);
    return item ? JSON.parse(item) : initialValue;
  });

  const setValue = (value: T) => {
    setStoredValue(value);
    localStorage.setItem(key, JSON.stringify(value));
  };

  return [storedValue, setValue] as const;
}
```

🛠️ TypeScript Tools in 2025 Industry

Tool	কাজ
tsx	TypeScript files directly run করে (ts-node replacement)

Tool	কাজ
Zod	Runtime type validation + TypeScript inference
tRPC	End-to-end type-safe API
TypeScript 5.x	Latest version — decorators, const type params

📖 TypeScript Resources

- **Official Docs:** https://www.typescriptlang.org/docs/
- **TypeScript Playground:** https://www.typescriptlang.org/play (browser-এ practice করো)
- **Total TypeScript (Matt Pocock):** https://www.totaltypescript.com (FREE exercises)
- **Execute Program:** https://www.executeprogram.com/courses/typescript

---

🎯 MISSION 2: Be a Node, Express & SQL SuperStar

তুমি ইতিমধ্যে Node.js ও Express.js জানো — এবার SQL শেখার পালা!

তুমি MongoDB (NoSQL) জানো। এবার **SQL (Relational Database)** শিখতে হবে। এটা Software Engineer হওয়ার জন্য অপরিহার্য।

SQL vs NoSQL — পার্থক্য বোঝো

বিষয়	SQL (PostgreSQL/MySQL)	NoSQL (MongoDB)
Data Structure	Tables (rows & columns)	Documents (JSON-like)
Schema	Fixed (defined upfront)	Flexible
Relations	JOIN দিয়ে	Embedding বা Reference
ACID	✅ Full support	Partial
Use Case	Banking, E-commerce	Real-time, Unstructured data

Real-world: Daraz, bKash, bank applications — সব SQL use করে। কারণ transactions-এ data integrity must।

📚 SQL Core Concepts

1. Database Design — সবার আগে এটা বোঝো

```sql
-- একটা simple e-commerce DB design

CREATE TABLE users (
    id SERIAL PRIMARY KEY,           -- Auto-increment
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE NOT NULL,
    created_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE products (
    id SERIAL PRIMARY KEY,
    name VARCHAR(200) NOT NULL,
    price DECIMAL(10, 2) NOT NULL,
    stock INT DEFAULT 0
);

CREATE TABLE orders (
    id SERIAL PRIMARY KEY,
    user_id INT REFERENCES users(id) ON DELETE CASCADE,
    total_amount DECIMAL(10, 2),
    status VARCHAR(50) DEFAULT 'pending',
    created_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE order_items (
    id SERIAL PRIMARY KEY,
    order_id INT REFERENCES orders(id),
    product_id INT REFERENCES products(id),
    quantity INT NOT NULL,
    price DECIMAL(10, 2) NOT NULL
);
```

2. CRUD Operations

```sql
```

```sql
-- CREATE
INSERT INTO users (name, email) VALUES ('Rahim Uddin', 'rahim@example.com');

-- READ
SELECT * FROM users WHERE email = 'rahim@example.com';

-- UPDATE
UPDATE products SET price = 1500.00 WHERE id = 1;

-- DELETE
DELETE FROM users WHERE id = 5;
```

3. JOINs — SQL-এর সবচেয়ে গুরুত্বপূর্ণ concept

```sql
-- একজন user-এর সব orders দেখাও (INNER JOIN)
SELECT
    u.name AS customer_name,
    o.id AS order_id,
    o.total_amount,
    o.status,
    o.created_at
FROM users u
INNER JOIN orders o ON u.id = o.user_id
WHERE u.id = 1;

-- Order details সহ products দেখাও
SELECT
    o.id AS order_id,
    p.name AS product_name,
    oi.quantity,
    oi.price,
    (oi.quantity * oi.price) AS subtotal
FROM orders o
INNER JOIN order_items oi ON o.id = oi.order_id
INNER JOIN products p ON oi.product_id = p.id
WHERE o.id = 100;
```

4. Aggregations — Analytics-এ অনেক কাজে লাগে

```sql
```

```sql
-- প্রতিটা user কত টাকার order করেছে
SELECT
    u.name,
    COUNT(o.id) AS total_orders,
    SUM(o.total_amount) AS total_spent,
    AVG(o.total_amount) AS avg_order_value
FROM users u
LEFT JOIN orders o ON u.id = o.user_id
GROUP BY u.id, u.name
HAVING COUNT(o.id) > 0
ORDER BY total_spent DESC;

-- বেস্ট সেলিং products
SELECT
    p.name,
    SUM(oi.quantity) AS total_sold
FROM products p
JOIN order_items oi ON p.id = oi.product_id
GROUP BY p.id, p.name
ORDER BY total_sold DESC
LIMIT 10;
```

5. Node.js + PostgreSQL (pg package)

```javascript
```

```javascript
// database.js
const { Pool } = require('pg');

const pool = new Pool({
  host: 'localhost',
  database: 'myapp',
  user: 'postgres',
  password: 'password',
  port: 5432,
});

// Query helper function
const query = (text, params) => pool.query(text, params);

module.exports = { query };

// userController.js
const db = require('./database');

async function getAllUsers(req, res) {
  try {
    const result = await db.query(
      'SELECT id, name, email, created_at FROM users ORDER BY created_at DESC'
    );
    res.json({ data: result.rows, total: result.rowCount });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
}

async function createUser(req, res) {
  const { name, email } = req.body;
  try {
    const result = await db.query(
      'INSERT INTO users (name, email) VALUES ($1, $2) RETURNING *',
      [name, email]  // Parameterized query — SQL injection থেকে safe
    );
    res.status(201).json({ data: result.rows[0] });
  } catch (error) {
    if (error.code === '23505') { // Unique violation
      return res.status(409).json({ error: 'Email already exists' });
    }
    res.status(500).json({ error: error.message });
  }
}
```

Middleware Chain Pattern

```javascript
// Real-world Express app structure
const express = require('express');
const app = express();

// Global middlewares
app.use(express.json());
app.use(cors());
app.use(helmet()); // Security headers
app.use(morgan('dev')); // Logging

// Auth middleware
const authenticate = async (req, res, next) => {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ error: 'No token' });

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decoded;
    next();
  } catch {
    res.status(401).json({ error: 'Invalid token' });
  }
};

// Role-based authorization
const authorize = (...roles) => (req, res, next) => {
  if (!roles.includes(req.user.role)) {
    return res.status(403).json({ error: 'Forbidden' });
  }
  next();
};

// Protected routes
app.get('/api/admin/users', authenticate, authorize('admin'), getAllUsers);
app.post('/api/products', authenticate, authorize('admin', 'seller'), createProduct)
```

📖 SQL & Node Resources

- **PostgreSQL Tutorial:** https://www.postgresqltutorial.com
- **SQLZoo (Interactive):** https://sqlzoo.net

- **Mode SQL Tutorial:** https://mode.com/sql-tutorial/
- **Use the Index, Luke (Advanced):** https://use-the-index-luke.com

---

🎯 MISSION 3: Be a RDBMS Rockstar

Relational Database Management System — গভীরে যাও

Mission 3-এ তুমি SQL শুধু জানবে না, master করবে।

Advanced SQL Concepts

1. Indexes — Performance এর চাবিকাঠি

```sql
-- Problem: 10 লক্ষ users আছে, email দিয়ে search করা slow
-- Solution: Index তৈরি করো

CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_orders_user_id ON orders(user_id);
CREATE INDEX idx_orders_status ON orders(status);

-- Composite index (multiple columns)
CREATE INDEX idx_orders_user_status ON orders(user_id, status);

-- EXPLAIN দিয়ে query performance দেখো
EXPLAIN ANALYZE SELECT * FROM users WHERE email = 'test@example.com';
```

2. Transactions — Data Integrity নিশ্চিত করো

```sql
-- Real-world: টাকা transfer করা (বিকাশ এরকম করে)
BEGIN;

UPDATE accounts SET balance = balance - 1000 WHERE user_id = 1;
UPDATE accounts SET balance = balance + 1000 WHERE user_id = 2;

-- যদি সব ঠিকঠাক থাকে
COMMIT;

-- যদি কোনো error হয়
-- ROLLBACK; -- সব পরিবর্তন বাতিল হয়ে যাবে
```

```javascript
// Node.js-ও Transaction
async function transferMoney(fromId, toId, amount) {
  const client = await pool.connect();

  try {
    await client.query('BEGIN');

    const senderResult = await client.query(
      'UPDATE accounts SET balance = balance - $1 WHERE user_id = $2 AND balance >=
      [amount, fromId]
    );

    if (senderResult.rowCount === 0) {
      throw new Error('Insufficient funds');
    }

    await client.query(
      'UPDATE accounts SET balance = balance + $1 WHERE user_id = $2',
      [amount, toId]
    );

    await client.query('COMMIT');
    return { success: true };

  } catch (error) {
    await client.query('ROLLBACK');
    throw error;
  } finally {
    client.release();
  }
}
```

3. Views, Stored Procedures & Triggers

```sql
```

```sql
-- View: Complex query কে simple করো
CREATE VIEW order_summary AS
SELECT
    o.id,
    u.name AS customer,
    COUNT(oi.id) AS item_count,
    SUM(oi.quantity * oi.price) AS total
FROM orders o
JOIN users u ON o.user_id = u.id
JOIN order_items oi ON o.id = oi.order_id
GROUP BY o.id, u.name;

-- ব্যবহার: Simple SELECT
SELECT * FROM order_summary WHERE total > 5000;

-- Trigger: Auto audit log তৈরি করো
CREATE TABLE audit_log (
    id SERIAL PRIMARY KEY,
    table_name VARCHAR(100),
    operation VARCHAR(10),
    old_data JSONB,
    new_data JSONB,
    changed_at TIMESTAMP DEFAULT NOW()
);

CREATE OR REPLACE FUNCTION log_user_changes()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO audit_log (table_name, operation, old_data, new_data)
    VALUES ('users', TG_OP, row_to_json(OLD), row_to_json(NEW));
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER user_audit_trigger
AFTER UPDATE OR DELETE ON users
FOR EACH ROW EXECUTE FUNCTION log_user_changes();
```

4. Database Normalization

```
1NF: প্রতিটা column-এ atomic value থাকবে (array নয়)
2NF: 1NF + partial dependency নেই
3NF: 2NF + transitive dependency নেই

Real-world example:
```

```
❌ Bad design:
orders table: id, customer_name, customer_email, product1, product2, product3

✅ Good design (3NF):
users: id, name, email
products: id, name, price
orders: id, user_id, created_at
order_items: id, order_id, product_id, quantity
```

🛠️ RDBMS Tools in Industry 2025

Tool	কাজ
PostgreSQL	Industry standard — সবার পছন্দ
MySQL/MariaDB	WordPress, legacy systems
TablePlus	GUI client (Mac/Windows)
DBeaver	Free GUI client
pgAdmin	PostgreSQL-এর official GUI
Supabase	PostgreSQL as a Service (Firebase alternative)

🎯 MISSION 4: Be a Prisma-Next.js Ninja

Prisma — Modern ORM যা TypeScript-এ কথা বলে

Prisma হলো একটা ORM (Object Relational Mapper) যা SQL queries লেখার বদলে JavaScript/TypeScript দিয়ে database operations করতে দেয়।

Prisma Setup & Schema

```bash
npm install prisma @prisma/client
npx prisma init
```

```prisma
```

```
// prisma/schema.prisma
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

model User {
  id        Int      @id @default(autoincrement())
  name      String
  email     String   @unique
  role      Role     @default(USER)
  posts     Post[]
  orders    Order[]
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
}

model Post {
  id        Int      @id @default(autoincrement())
  title     String
  content   String?
  published Boolean  @default(false)
  author    User     @relation(fields: [authorId], references: [id])
  authorId  Int
  createdAt DateTime @default(now())
}

enum Role {
  USER
  ADMIN
  MODERATOR
}
```

Prisma CRUD Operations

```typescript
```

```
import { PrismaClient } from '@prisma/client';
const prisma = new PrismaClient();

// CREATE
const newUser = await prisma.user.create({
  data: {
    name: "Farhan Ahmed",
    email: "farhan@example.com",
    posts: {
      create: {  // Nested create — User সাথে Post তৈরি
        title: "My First Post",
        content: "Hello World"
      }
    }
  },
  include: { posts: true }  // Response-এ posts include করো
});

// READ with filtering
const activeAdmins = await prisma.user.findMany({
  where: {
    role: 'ADMIN',
    createdAt: {
      gte: new Date('2024-01-01')  // January 2024 এর পর তৈরি
    }
  },
  orderBy: { createdAt: 'desc' },
  take: 10,   // LIMIT
  skip: 20,   // OFFSET (pagination)
  select: {   // শুধু এই fields নাও (performance!)
    id: true,
    name: true,
    email: true,
    _count: { select: { posts: true } }  // posts count
  }
});

// UPDATE
const updated = await prisma.user.update({
  where: { id: 1 },
  data: { name: "Farhan Rahman" }
});

// DELETE (Cascade)
await prisma.user.delete({ where: { id: 5 } });
```

```
// Transaction
const [updatedSender, updatedReceiver] = await prisma.$transaction([
  prisma.account.update({
    where: { userId: 1 },
    data: { balance: { decrement: 1000 } }
  }),
  prisma.account.update({
    where: { userId: 2 },
    data: { balance: { increment: 1000 } }
  })
]);
```

Next.js Advanced (তুমি basic জানো — এবার এগিয়ে যাও)

```typescript
```

```
// app/api/users/route.ts — App Router API
import { NextRequest, NextResponse } from 'next/server';
import { prisma } from '@/lib/prisma';

export async function GET(request: NextRequest) {
  const searchParams = request.nextUrl.searchParams;
  const page = Number(searchParams.get('page') || 1);
  const limit = 10;

  const [users, total] = await Promise.all([
    prisma.user.findMany({
      skip: (page - 1) * limit,
      take: limit,
      select: { id: true, name: true, email: true, role: true }
    }),
    prisma.user.count()
  ]);

  return NextResponse.json({
    data: users,
    pagination: { page, limit, total, pages: Math.ceil(total / limit) }
  });
}

// Server Component (No useEffect needed!)
// app/users/page.tsx
async function UsersPage() {
  const users = await prisma.user.findMany(); // Direct DB call from component!

  return (
    <div>
      {users.map(user => <UserCard key={user.id} user={user} />)}
    </div>
  );
}
```

📖 Prisma & Next.js Resources

- **Prisma Docs:** https://www.prisma.io/docs
- **Prisma + Next.js Guide:** https://www.prisma.io/nextjs
- **Next.js Official Docs:** https://nextjs.org/docs
- **Josh tried coding (YouTube):** Best Next.js tutorials

🎯 MISSION 5 PREVIEW: Be a Backend Brainiac

Advanced Backend Concepts — এগুলো explore করো

REST API Best Practices

```
✅ Proper HTTP Methods: GET, POST, PUT, PATCH, DELETE
✅ Meaningful Status Codes: 200, 201, 400, 401, 403, 404, 409, 500
✅ Versioning: /api/v1/users
✅ Pagination: ?page=1&limit=10
✅ Filtering: ?status=active&role=admin
✅ Error Handling: Consistent error response format
```

JWT Authentication Pattern

typescript

```typescript
// Auth service
class AuthService {
  async login(email: string, password: string) {
    const user = await prisma.user.findUnique({ where: { email } });
    if (!user) throw new Error('User not found');

    const isValid = await bcrypt.compare(password, user.password);
    if (!isValid) throw new Error('Invalid password');

    const accessToken = jwt.sign(
      { userId: user.id, role: user.role },
      process.env.JWT_SECRET!,
      { expiresIn: '15m' }  // Short-lived
    );

    const refreshToken = jwt.sign(
      { userId: user.id },
      process.env.REFRESH_SECRET!,
      { expiresIn: '7d' }  // Long-lived
    );

    return { accessToken, refreshToken };
  }
}
```

🎯 MISSION 6 PREVIEW: Be a Frontend Futurist

তুমি React জানো — এবার এগুলো শেখো

Advanced React Patterns

- **React Query / TanStack Query** — Server state management
- **Zustand** — Lightweight global state (Redux alternative)
- **React Hook Form + Zod** — Form validation with TypeScript

```typescript
// React Query example
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query';

function UserList() {
  const queryClient = useQueryClient();

  const { data: users, isLoading } = useQuery({
    queryKey: ['users'],
    queryFn: () => fetch('/api/users').then(r => r.json())
  });

  const deleteMutation = useMutation({
    mutationFn: (id: number) => fetch(`/api/users/${id}`, { method: 'DELETE' }),
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ['users'] }); // Auto refresh!
    }
  });

  if (isLoading) return <Spinner />;

  return users.map(user => (
    <div key={user.id}>
      {user.name}
      <button onClick={() => deleteMutation.mutate(user.id)}>Delete</button>
    </div>
  ));
}
```

🎯 MISSION 7 PREVIEW: Docker & Nginx Nomad

Docker — "It works on my machine" সমস্যার সমাধান

```dockerfile
# Dockerfile — Node.js app containerize করো
FROM node:20-alpine

WORKDIR /app

COPY package*.json ./
RUN npm ci --only=production

COPY . .
RUN npm run build

EXPOSE 3000
CMD ["node", "dist/server.js"]
```

```yaml
# docker-compose.yml — Multiple services একসাথে চালাও
version: '3.8'
services:
  app:
    build: .
    ports: ["3000:3000"]
    environment:
      DATABASE_URL: postgresql://postgres:password@db:5432/myapp
    depends_on: [db]

  db:
    image: postgres:16-alpine
    volumes: [postgres_data:/var/lib/postgresql/data]
    environment:
      POSTGRES_DB: myapp
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: password

volumes:
  postgres_data:
```

🎯 MISSION 8 PREVIEW: AI Essentials for Full Stack Developers

AI Integration — 2025-এর সবচেয়ে hot skill

```typescript
// OpenAI / Anthropic API integration
import Anthropic from '@anthropic-ai/sdk';

const client = new Anthropic({ apiKey: process.env.ANTHROPIC_API_KEY });

// Real-world: AI-powered customer support
async function getAISupport(userQuestion: string) {
  const message = await client.messages.create({
    model: "claude-opus-4-5",
    max_tokens: 1024,
    messages: [{ role: "user", content: userQuestion }],
    system: "You are a helpful customer support agent for an e-commerce platform."
  });

  return message.content[0].text;
}

// Vercel AI SDK — Next.js-এ সহজে AI integrate করো
import { streamText } from 'ai';
import { openai } from '@ai-sdk/openai';

export async function POST(req: Request) {
  const { messages } = await req.json();

  const result = streamText({
    model: openai('gpt-4o'),
    messages,
  });

  return result.toDataStreamResponse();
}
```

📅 2-WEEK STUDY PLAN (Before May 1, 2026)

Week 1: TypeScript Mastery (Apr 18–24)

দিন	Topic	সময়	Resource
Day 1	TS Setup, Basic Types, Type Annotations	2-3 hrs	TypeScript Playground
Day 2	Interfaces, Type Aliases, Union/Intersection	2-3 hrs	Total TypeScript
Day 3	Generics — Functions, Classes, Interfaces	3 hrs	TS Official Docs
Day 4	Utility Types, Enums, Decorators	2-3 hrs	Matt Pocock videos
Day 5	TypeScript + React (Props, Hooks, Events)	3 hrs	Build a typed project
Day 6	TypeScript + Node.js + Express	3 hrs	Build REST API
Day 7	**Project:** Typed Todo App (React + Express + TS)	4-5 hrs	Portfolio project

Week 2: SQL + Database Fundamentals (Apr 25–30)

দিন	Topic	সময়	Resource
Day 8	PostgreSQL setup, DDL (CREATE, ALTER, DROP)	2-3 hrs	PostgreSQL Tutorial
Day 9	DML (SELECT, INSERT, UPDATE, DELETE)	2-3 hrs	SQLZoo interactive
Day 10	JOINs (INNER, LEFT, RIGHT, FULL)	3 hrs	Mode SQL Tutorial
Day 11	Aggregations, GROUP BY, HAVING, Subqueries	2-3 hrs	Practice queries
Day 12	Indexes, Transactions, Views	3 hrs	PostgreSQL docs
Day 13	Prisma Setup, Schema design, CRUD	3 hrs	Prisma docs
Day 14	**Project:** Full-stack app with Next.js + Prisma + PostgreSQL	5 hrs	Portfolio project

---

🛠️ Tools & Software Install করো এখনই

```bash
```

```
# 1. Node.js (LTS)
https://nodejs.org

# 2. PostgreSQL
https://www.postgresql.org/download/

# 3. TablePlus or DBeaver (DB GUI)
https://tableplus.com

# 4. VS Code Extensions
- Prisma (Prisma.prisma)
- TypeScript Hero
- REST Client (or use Thunder Client)
- Error Lens
- Pretty TypeScript Errors

# 5. Git & GitHub — আগে থেকেই জানো, কিন্তু নিশ্চিত করো সব কিছু set up আছে
```

💡 Pro Tips — একজন 7+ বছরের Instructor হিসেবে পরামর্শ

1. **প্রতিদিন code করো** — theory পড়ার চেয়ে hands-on practice বেশি গুরুত্বপূর্ণ
2. **Projects বানাও** — প্রতিটা concept শেখার পর একটা ছোট project বানাও
3. **Error পড়তে শেখো** — TypeScript errors ভয়ের না, তারা তোমার friend
4. **GitHub-এ push করো** — সব project GitHub-এ রাখো, portfolio হবে
5. **Course-এর আগে সব জানতে হবে না** — মূল concepts clear করলেই হবে
6. **Community join করো** — Programming Hero-র Discord/Facebook group-এ active থাকো

🎯 তোমার Strength Points (ইতিমধ্যে যা জানো)

```
✅ React → TypeScript + React শেখা সহজ হবে
✅ Node.js + Express → Advanced patterns শিখবে
✅ MongoDB → SQL-এর contrast বুঝবে সহজে
✅ Firebase → Authentication patterns জানা আছে
✅ Next.js Basic → Advanced Next.js শিখতে সুবিধা
✅ Tailwind → Frontend Futurist mission সহজ হবে
```

> **মনে রাখো:** তুমি একজন Frontend Developer থেকে Software Engineer হচ্ছো। এই journey টা exciting। ভয় পেও না — তোমার base অনেক strong। শুধু SQL আর TypeScript-এ focus করো।

> বাকিটা course-এই শিখবে।
> **শুভকামনা!** 🚀
> *May 1, 2026 থেকে শুরু হোক তোমার Software Engineer journey!*